VC_FIFO

Overview

VC_FIFO is a general-purpose verification pattern, implementing a parametrical FIFO functionality. This component is useful for validation of various store-and-forward devices. Upon a FIFO overflow or FIFO underflow, the FIFO enters a “wait” mode, waiting for another push or pull transaction which will bring the FIFO into the normal regime.

Syntax

```
vc_fifo #(bit_width, num_words, fifo_depth) InstName ();
```

Parameters		
bit_width		Bit width of the data input word
num_words		Maximum number of words for input transaction
Fifo_depth		Maximum FIFO depth
Access Tasks		
Name	*Arguments*	*Description*
reset	No	Resets fifo, setting both read and write counters to 0
set_data	word_number data_word	- The number of data word to be set (must be lower than the “num_words” parameter) - The value of a defined data word Example: `for (i=0; i<4; i=i+1) fifo.set_data(i,1);` -set four first words of fifo entry to be equal to 1

push	No	Push data into FIFO; increments wptr pointer. Must be used after “set_data”.
		
		Example:
		`for (i=0; i<4; i=i+1) fifo.set_data(i,1);`
		`fifo.push;`

pull	No	Pull data from FIFO into the “data_out” access register; increments rptr pointer.
		
		Example:
		`for (i=0; i<4; i=i+1) fifo.set_data(i,1);`
		`fifo.push;`
		`fifo.pull;`
		`for (i=0; i<fifo.data_size; i=i+1)`
		`  $display(“data(%0d) = %h”, I, fifo.data_out(i));`

Access Registers		
Data_out	This register contains data which is pushed out from FIFO. The size of the register is defined by a “bit_width” parameter	
Data_size	Contains the number of words for the output transaction. Must be used in conjunction with data_out to read_out pushed data.	
Access Variables		
rptr	Read pointer	
wptr	Write pointer	

Verilog example

The example below demonstrates the use of a vc_fifo component.

```
module test;

reg clk;
initial clk = 0;
always #5 clk <= ~clk;

integer i;

//------- WLEN NUMW DEPTH ----------------
vc_fifo #(8,   4,   4)  fifo ();

initial begin
  $dumpvars;
//  $monitor("wptr = %0d, rptr = %0d, dist = %0d, time = %0t",
//    fifo.wptr, fifo.rptr, fifo.dist_ptr, $time);
  #1000 $finish;
end

initial begin
for (i=0; i<3; i=i+1) fifo.set_data(i,1);
#10  fifo.push;
for (i=0; i<2; i=i+1) fifo.set_data(i,2);
#10  fifo.push;
for (i=0; i<4; i=i+1) fifo.set_data(i,3);
#10  fifo.push;
for (i=0; i<1; i=i+1) fifo.set_data(i,4);
#10  fifo.push;
for (i=0; i<2; i=i+1) fifo.set_data(i,5);
#10  fifo.push;
for (i=0; i<3; i=i+1) fifo.set_data(i,6);
#10  fifo.push;
for (i=0; i<3; i=i+1) fifo.set_data(i,7);
#10  fifo.push;
for (i=0; i<3; i=i+1) fifo.set_data(i,8);
#10  fifo.push;
end

initial begin
#100 fifo.pull;
#10 fifo.pull;
#10 fifo.pull;
#10 fifo.pull;
#10 fifo.pull;

#100 $finish;
end

endmodule
```